

Fake News Detection

Group: Group 2B

Project

1. Introduction

The digital age is characterized by the rapid and widespread dissemination of information. The dissemination of news articles with implausible, dubious, dishonest or unreliable content propagates misinformation, erodes public trust, and manipulates public opinion. Manual fact-checking is unfeasible and unviable owing to the sheer volume of "informational textual content" posted across news dissemination websites and shared across social media platforms. This project presents a robust machine learning framework designed to automatically detect fake news by classifying news articles as either genuine (Class 1) or fabricated (Class 0). The methodology centers on natural language processing (NLP), Logistic Regression, Ensemble models and Deep Learning to analyze the linguistic features, sentiment, and contextual features in text data.

1.1 Business Understanding

In today's information-driven society, the spread of misinformation and fake news through online news dissemination platforms and social media platforms has become a serious threat to public trust, democracy, health communication, and media integrity. The ability to automatically distinguish fake from real news is essential for:

- Social media platforms (e.g., Meta, Twitter/X, Reddit).
- Fact-checking organizations.
- News aggregators (e.g., Google News, Apple News).
- The general public.

1.2 Problem Statement:

The proliferation of fake news on digital platforms poses a significant risk to public trust and decision-making. Traditional methods of identifying fake news are slow and resource-intensive, making it difficult to manage the vast volume of online content.

1.3 Proposed solution

This project aims to develop an automated binary classification system using transformer-based Natural Language Processing (NLP) models to detect and flag fake news articles with high accuracy. By leveraging advanced NLP techniques, the system will offer a scalable solution to help identify harmful misinformation, improving information credibility and supporting the fight against digital manipulation.

1.4 Objectives

1. Perform Extrapolatory Data Analysis.

2. Build a Baseline Logistic Regression Model to predict class of an article.
3. Build an XGBoost model, a LSTM neural network and finetune the RoBERTa-base Transformer.
4. Interpret best performing ML model using the LIME library.
5. Deploy selected model using FastAPI and Streamlit.

1.5 Success Criteria

This project aims to catch the majority of fake news articles based on respective text (title + "" + scrapped_article_text). However, the classes for the target variable of the dataset used in this project is imbalanced. Thus, the **macro-averaged recall** metric (the average recall across all classes, treating each class equally) is set as the success criteria. Macro-averaged recall is particularly important when dealing with imbalanced datasets because it ensures that performance on the minority class is not overshadowed by the majority class. We aim to achieve a **macro-averaged recall** of over **76%** for the deployed model. Additionally, we aim to achieve an **accuracy** of at least **80%** and an **AUC** of over **0.8** on the test set.

2. Data Loading, Feature Engineering, and Preprocessing

2.1 Data Loading

```
In [1]: ▶ # Declare dataset path
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

/kaggle/input/news-article-credibility-detection-dataset/data.csv

In [2]: ▶ # Declare images directory
import os
images_dir = '/kaggle/working/images'
os.makedirs(images_dir, exist_ok=True)
print(os.listdir('/kaggle/working/'))

['images', '.virtual_documents']
```



```
In [2]: # Import basic libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
import io

# Import NLP tools and NLTK library modules
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk import pos_tag
from nltk.util import bigrams
from nltk.util import trigrams
import string
import re
import spacy

# Import scikit-learn library's classes, tools, and modules
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.metrics import accuracy_score, classification_report,
from sklearn.metrics import f1_score, precision_score, recall_score
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.utils.class_weight import compute_class_weight
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import RandomUnderSampler
import joblib
from gensim.models import Word2Vec

# Import tensorflow, and keras modules
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Model, load_model
from tensorflow.keras.layers import TextVectorization, Embedding, Conv1D,
from tensorflow.keras.layers import Dropout, Dense, LSTM, Input, MaxPooling1D
from tensorflow.keras.metrics import Precision, Recall, AUC
from transformers import TFRobertaForSequenceClassification, RobertaTokenizerFast
from transformers import logging as transformers_logging

# Import transformers and pytorch modules
from transformers import RobertaTokenizerFast, RobertaForSequenceClassification
from datasets import Dataset
import torch
from torch.utils.data import Dataset
import os

# Import Model Interpretation Libraries and modules
from lime.lime_text import LimeTextExplainer
import matplotlib.patches as mpatches
```

```
# Disable warnings
import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: path = './dataset/data.csv'
data = pd.read_csv(path)
data.head()
```

Out[3]:

	news_url	title	extracted_article_text	news_type
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip
2	https://www.unitedbypop.com/style/fashion/tayl...	Outfit ideas for Taylor Swift's Reputation Tour	United By Pop - United Kingdom. United States....	gossip
3	https://variety.com/2018/music/news/scott-hutc...	Scott Hutchison Dead: Frightened Rabbit Frontm...	By Robert Mitchell A body found Thursday night...	gossip
4	https://www.etonline.com/kate-middleton-gives-...	Kate Middleton Gives Birth to Royal Baby No. 3...	Baby No. 3 is officially here! Kate Middleton ...	gossip

```
In [4]: # Check shape
data.shape
```

Out[4]: (16044, 5)

```
In [5]: # Check duplicates
data.duplicated().sum()
```

Out[5]: 928

```
In [6]: # Drop duplicates
data.drop_duplicates(inplace=True)
```

```
In [7]: # Confirm no duplicates
data.duplicated().sum()
```

Out[7]: 0

In [8]: `# Inspect column attributes to check missing values`

```
data.info()

<class 'pandas.core.frame.DataFrame'>
Index: 15116 entries, 0 to 16043
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   news_url              15116 non-null  object
 1   title                 15116 non-null  object
 2   extracted_article_text 15116 non-null  object
 3   news_type             15116 non-null  object
 4   class                 15116 non-null  int64
dtypes: int64(1), object(4)
memory usage: 708.6+ KB
```

- Data was successfully loaded and duplicates removed.
- There are no entries with missing values

2.2 Feature Engineering

In [9]: `# Engineer the domain feature`
`from urllib.parse import urlparse`
`data['domain'] = data['news_url'].apply(lambda x: urlparse(x).netloc)`

`# Engineer the article feature`
`data['article'] = data['title'] + " " + data['extracted_article_text']`

`# Engineer the text length`
`data['num_sentences'] = data['extracted_article_text'].apply(lambda x: len(x.split('.')))`

`# Engineer the text length`
`data['text_length'] = data['extracted_article_text'].apply(lambda x: len(x))`
`data.head(2)`

Out[9]:

	news_url	title	extracted_article_text	news_type	cla
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip	
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip	

```
In [10]: data['domain'].value_counts().nlargest(5)
```

```
Out[10]: domain
missing                3237
people.com             1224
www.dailymail.co.uk    682
www.etimes.com         536
www.usmagazine.com     512
Name: count, dtype: int64
```

2.3 Data Preprocessing

- Define a function for **text cleaning, normalization, word tokenizing, Part-of-Speech tagging, lemmatization** and **removing stopwords**, using the **spaCy** library.

```

In [11]: ► # Load spaCy model
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])

# Define function to clean text using spaCy
def cleaned_text(text, debug=False):

    # Convert lists to strings
    if isinstance(text, list):
        text = " ".join(text)

    # Ensure text is a string
    if not isinstance(text, str):
        text = str(text)

    # Lowercase all characters, remove URLs, enclosed by [] <>, &
    text = text.lower()
    text = re.sub(r'https?://\S+|www\.\S+', '', text)
    text = re.sub(r'\[.*?\]', '', text)
    text = re.sub(r'<.*?>+', '', text)
    text = re.sub(r'\n', ' ', text)

    # Define chunk size
    chunk_size = 900000
    cleaned_tokens = []

    # Process text in chunks
    if len(text) > chunk_size:
        chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]
    else:
        chunks = [text]

    for chunk in chunks:
        doc = nlp(chunk)

        # Extract lemmatized tokens, excluding stopwords, and punctuation
        for token in doc:
            if (not token.is_stop and
                not token.is_punct):
                cleaned = re.sub(r'^a-zA-Z', '', token.lemma_)

                # Only include non-empty strings
                if cleaned:
                    cleaned_tokens.append(cleaned)

    return cleaned_tokens

```

```

In [12]: ► # Apply the clean_text function and create the 'spacy' column to hold cleaned text
data['spacy'] = data['article'].apply(cleaned_text)

```



```
In [13]: # Join tokens
data['text'] = data['spacy'].apply(lambda x: ' '.join(x))
data.head(2)
```

Out[13]:

	news_url	title	extracted_article_text	news_type	cla
0	https://www.bustle.com/p/will-the-royals-retur...	Will 'The Royals' Return For Season 5? This St...	Entertainment With a royal wedding and a coup ...	gossip	
1	https://www.foxnews.com/entertainment/naya-riv...	Naya Rivera refiles for divorce from Ryan Dors...	This material may not be published, broadcast,...	gossip	

```
In [14]: # Preview a random sample
data.iloc[99, 10]
```

Out[14]: 'blac chyna cozy hot felon jeremy meek pic look like blac chyna h
ot felon jeremy meek work chyna share picture posing arm snapchat
wednesday night show tattoo chyna don skintight lace orange dress
meek keep casual camouflage print shirt rip jean news early snapc
hat post chyna appear photo shoot lead speculate possibly star ca
mpaign meek model attractive mugshot go viral headline summer rel
ationship topshop heiress chloe green meek photograph kiss green
luxury yacht coast turkey july married wife melissa time shortly
picture surface meek file separation melissa year marriage news m
eek green shy pack pda watch video'

```
In [15]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 15116 entries, 0 to 16043
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   news_url              15116 non-null  object
1   title                 15116 non-null  object
2   extracted_article_text 15116 non-null  object
3   news_type             15116 non-null  object
4   class                 15116 non-null  int64
5   domain                15116 non-null  object
6   article                15116 non-null  object
7   num_sentences          15116 non-null  int64
8   text_length            15116 non-null  int64
9   spacy                  15116 non-null  object
10  text                   15116 non-null  object
dtypes: int64(3), object(8)
memory usage: 1.4+ MB
```

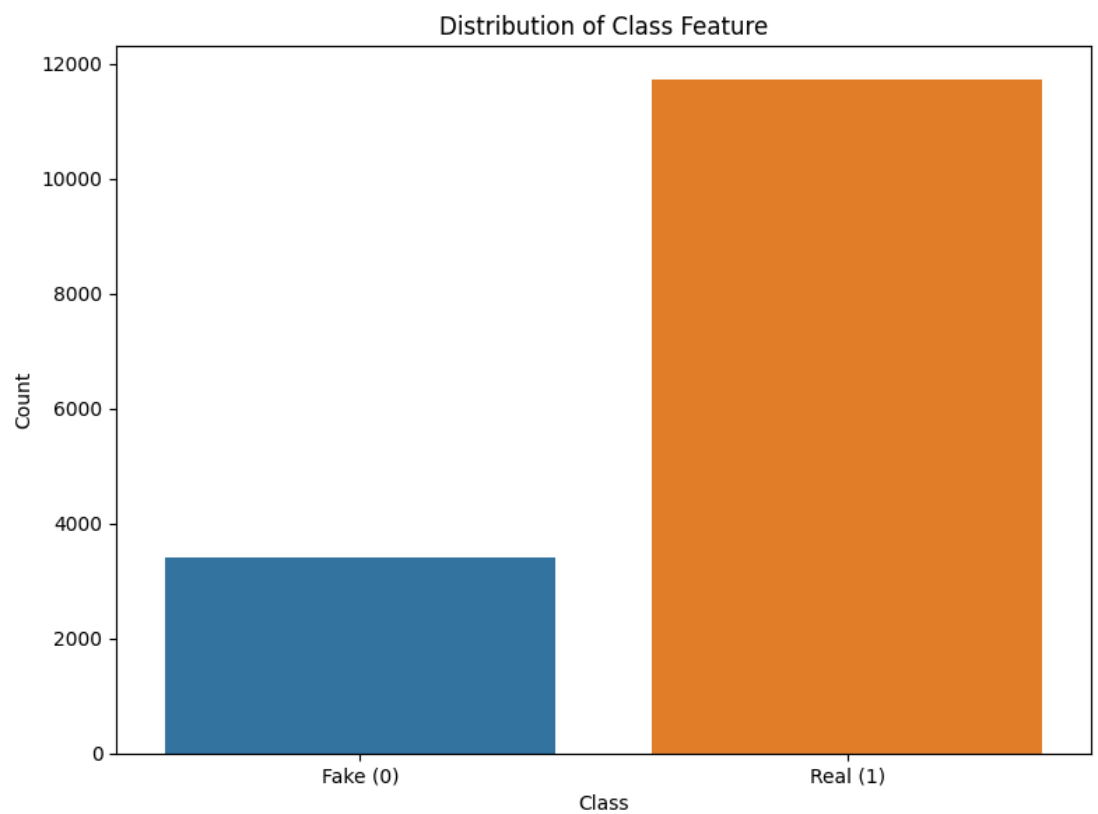
Objective 1: Perform Extraplorary Data Analysis

Target Variable Class Distribution

```
In [16]: ▶ # Print value counts for each class  
data['class'].value counts()
```

```
Out[16]: class  
1      11717  
0       3399  
Name: count, dtype: int64
```

```
In [25]: ▶ # Visualize Class Distribution  
plt.figure(figsize=(8, 6))  
sns.barplot(x='class', y='class', data=data, estimator=len)  
plt.xlabel('Class')  
plt.ylabel('Count')  
plt.title('Distribution of Class Feature')  
plt.xticks([0, 1], ['Fake (0)', 'Real (1)'])  
plt.tight_layout()  
plt.savefig('/kaggle/working/images/class-distribuctions.png', dpi=  
plt.show()
```



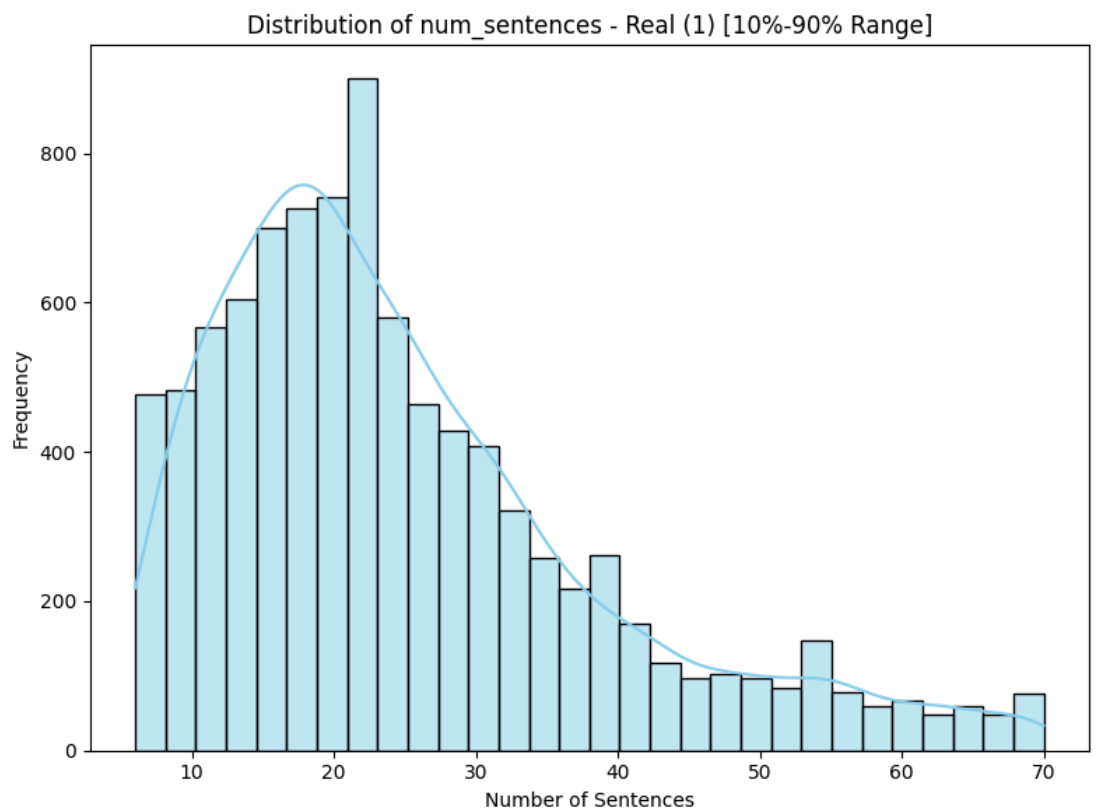
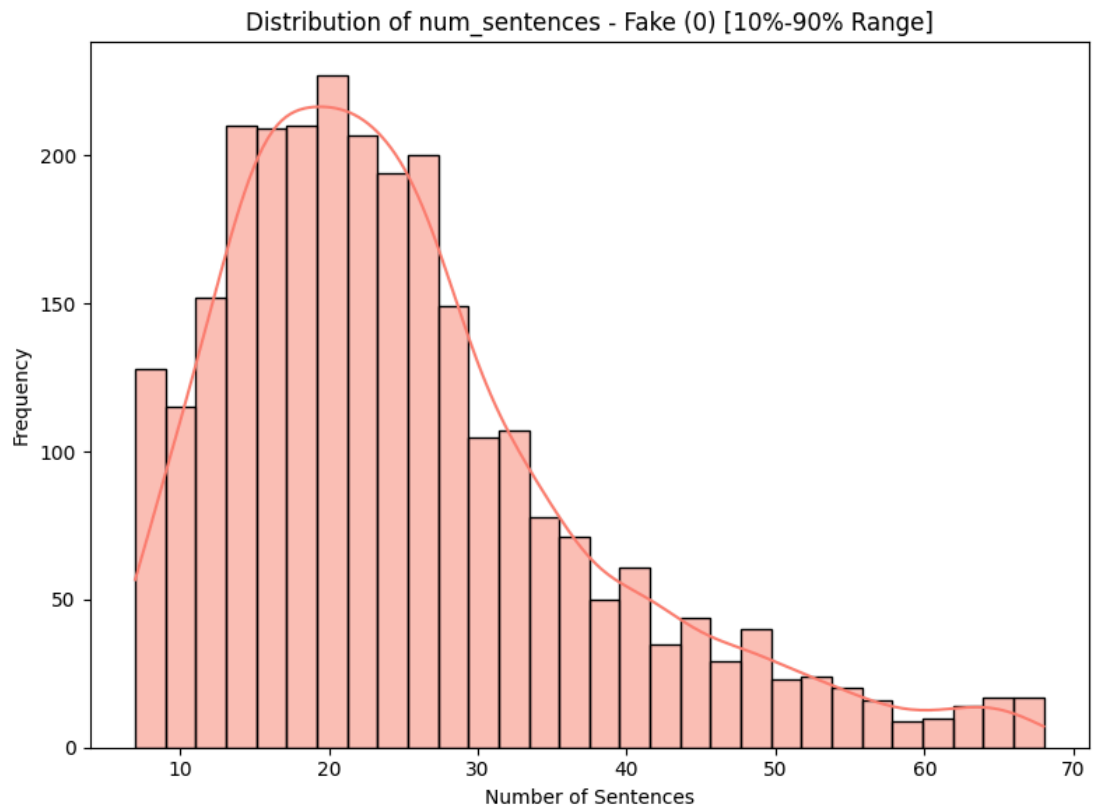
num_sentences Distribution

```
In [22]: ▶ # Visualize num_sentences distributions

# Function to filter between 10% and 90% percentiles to avoid skew
def filter_percentile(df, col):
    lower = df[col].quantile(0.10)
    upper = df[col].quantile(0.90)
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Plot distribution for class 0 (filtered data)
class0_data = filter_percentile(data[data['class'] == 0], 'num_sentences')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class0_data,
    x='num_sentences',
    kde=True,
    bins=30,
    color='salmon')
plt.title('Distribution of num_sentences - Fake (0) [10%-90% Range]')
plt.xlabel('Number of Sentences')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/num_sentences_distribution_class0_filtered.png')
plt.show()

# Plot distribution for class 1 (filtered data)
class1_data = filter_percentile(data[data['class'] == 1], 'num_sentences')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class1_data,
    x='num_sentences',
    kde=True,
    bins=30,
    color='skyblue')
plt.title('Distribution of num_sentences - Real (1) [10%-90% Range]')
plt.xlabel('Number of Sentences')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/num_sentences_distribution_class1_filtered.png')
plt.show()
```



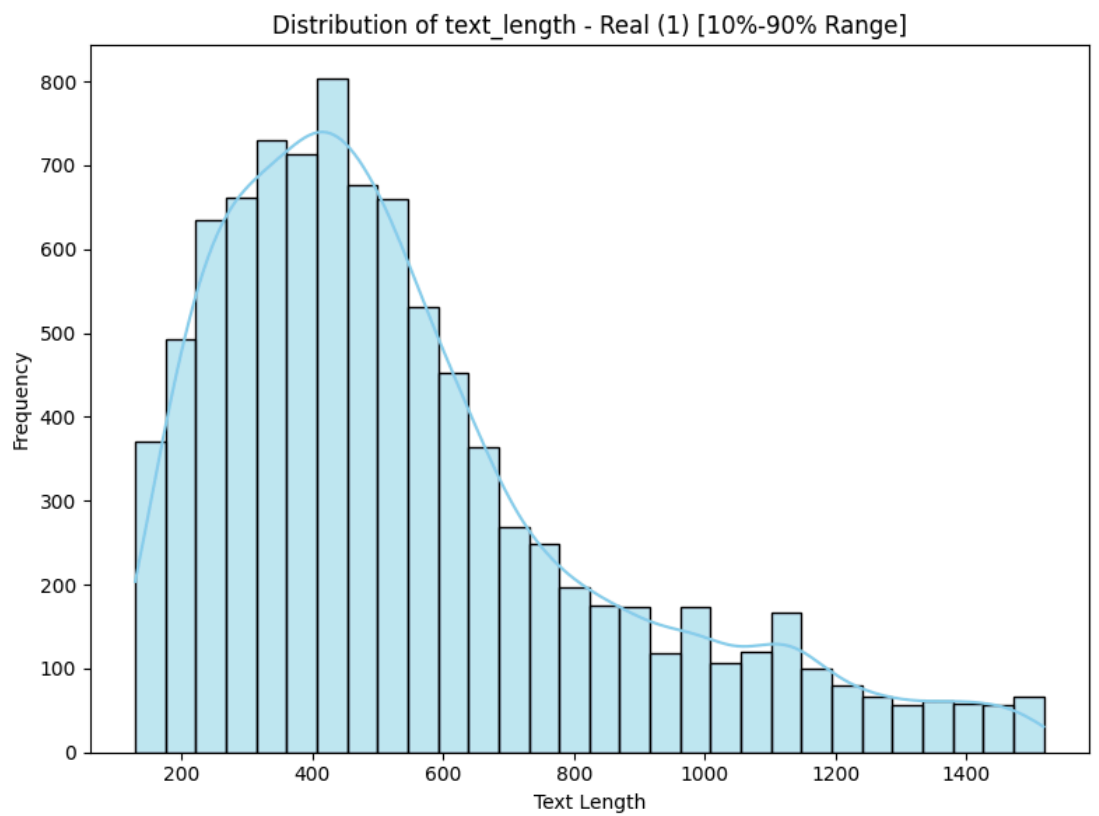
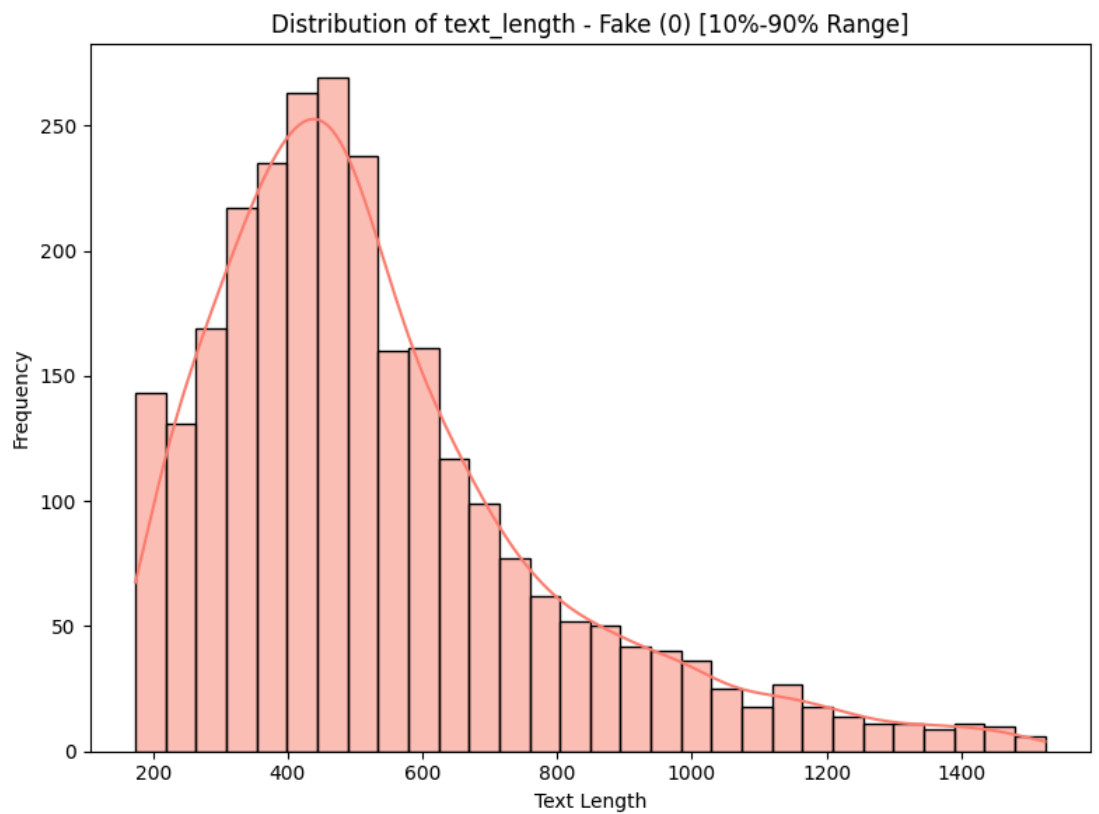
text_length Distribution

```
In [23]: ▶ # Visualize num_sentences distributions

# Function to filter between 10% and 90% percentiles to avoid skew
def filter_percentile(df, col):
    lower = df[col].quantile(0.10)
    upper = df[col].quantile(0.90)
    return df[(df[col] >= lower) & (df[col] <= upper)]

# Plot distribution for class 0 (filtered data)
class0_data = filter_percentile(data[data['class'] == 0], 'text_length')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class0_data,
    x='text_length',
    kde=True,
    bins=30,
    color='salmon')
plt.title('Distribution of text_length - Fake (0) [10%-90% Range]')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/text_length_distribution_class0_filtered.png')
plt.show()

# Plot distribution for class 1 (filtered data)
class1_data = filter_percentile(data[data['class'] == 1], 'text_length')
plt.figure(figsize=(8, 6))
sns.histplot(
    data=class1_data,
    x='text_length',
    kde=True,
    bins=30,
    color='skyblue')
plt.title('Distribution of text_length - Real (1) [10%-90% Range]')
plt.xlabel('Text Length')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('./images/text_length_distribution_class1_filtered.png')
plt.show()
```



Top-5 Domains for Fake and Real News Articles

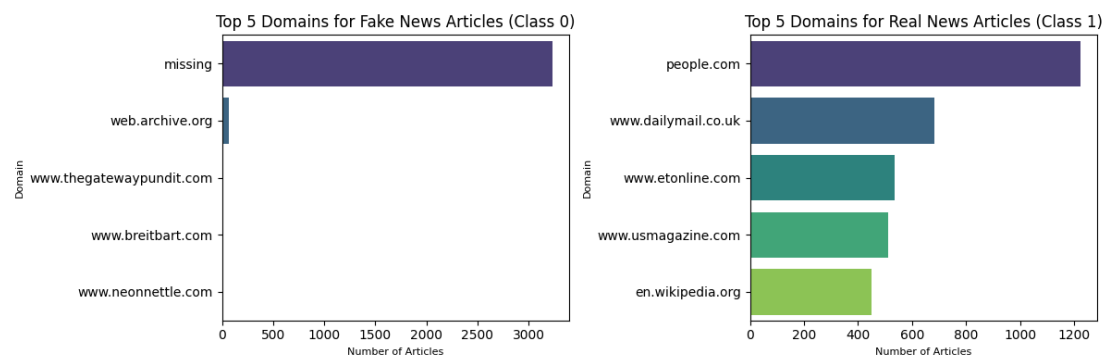
```

In [26]: ▶ # Visualize top 5 domains for fake news and real news
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# Plot for class 0 (Fake News)
fake_data = data[data['class'] == 0]
domain_counts = fake_data['domain'].value_counts()
top_5_domains = domain_counts.nlargest(5)
sns.barplot(x=top_5_domains.values, y=top_5_domains.index, palette=
ax1.set_title('Top 5 Domains for Fake News Articles (Class 0)', for
ax1.set_xlabel('Number of Articles', fontsize=8)
ax1.set_ylabel('Domain', fontsize=8)

# Plot for class 1 (Real News)
fake_data = data[data['class'] == 1]
domain_counts = fake_data['domain'].value_counts()
top_5_domains = domain_counts.nlargest(5)
sns.barplot(x=top_5_domains.values, y=top_5_domains.index, palette=
ax2.set_title('Top 5 Domains for Real News Articles (Class 1)', for
ax2.set_xlabel('Number of Articles', fontsize=8)
ax2.set_ylabel('Domain', fontsize=8)
plt.tight_layout()
plt.savefig('/kaggle/working/images/top-5-domains-for-fake-and-real
plt.show()

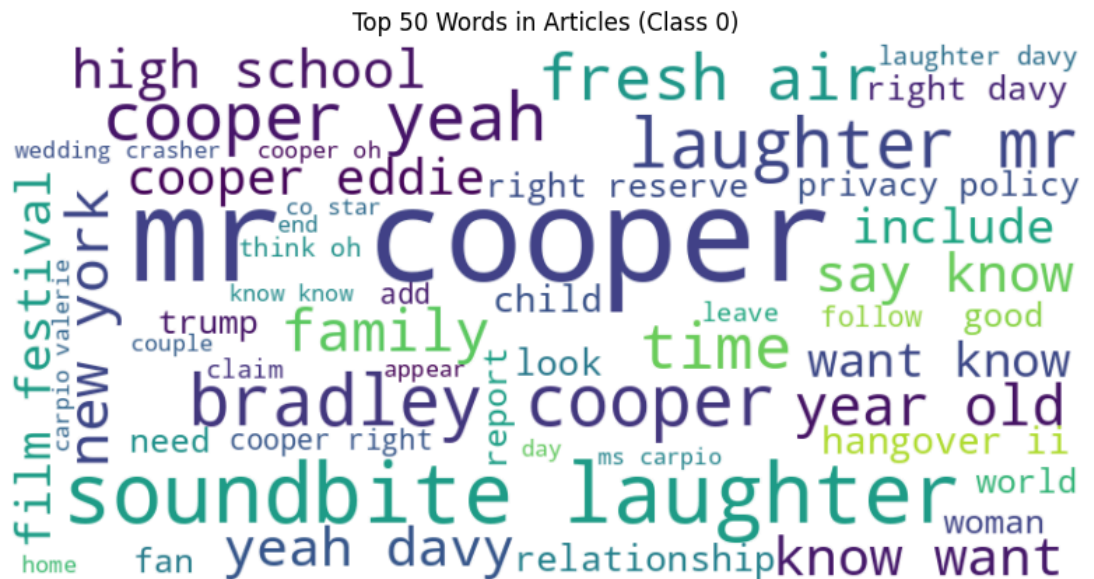
```



Most Frequent/ Significant Words Among Fake News Articles (Label 0)

```
In [27]: ▶ # Filter data for class 0
class_0_data = data[data['class'] == 0]

# Combine all text from extracted_article_text
text = ' '.join(class_0_data['text'].dropna().astype(str))
wordcloud = WordCloud(width=800, height=400, background_color='white')
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Top 50 Words in Articles (Class 0)')
plt.savefig('/kaggle/working/images/top-50-words-for-class0-wordcloud.png')
plt.show()
```



Most Frequent/ Significant Words Among Real News Articles (Label 1)


```

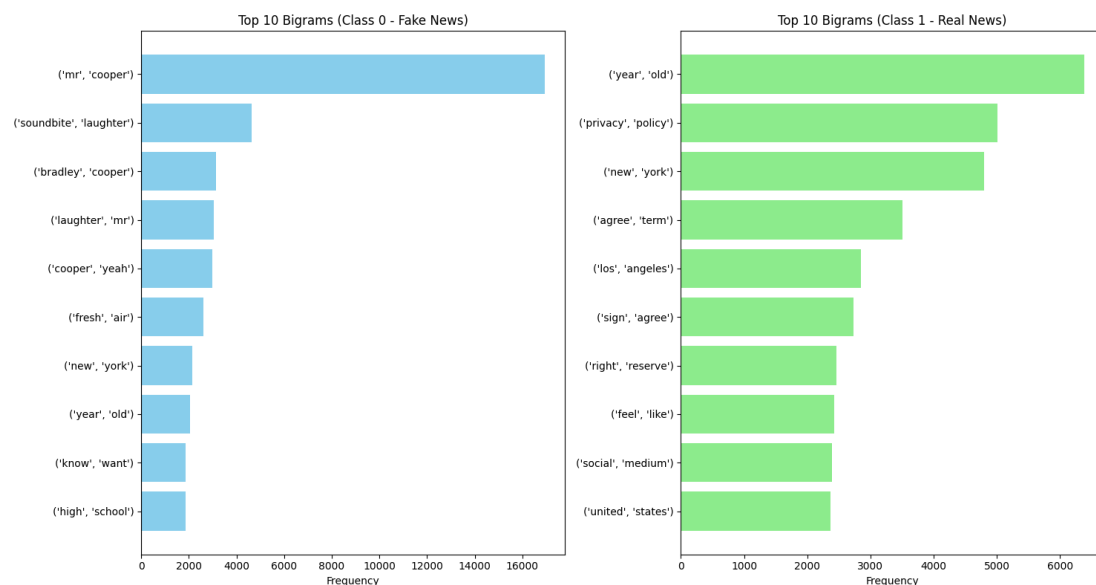
In [29]: # Filter and plot top-10 bigrams for both fake and real news articles
def get_top_bigrams(text_series):
    all_bigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        bigram_list = list(bigrams(tokens))
        all_bigrams.extend(bigram_list)
    return Counter(all_bigrams).most_common(10)

class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']

# Get top 10 bigrams for each class
top_bigrams_class_0 = get_top_bigrams(class_0_data)
top_bigrams_class_1 = get_top_bigrams(class_1_data)
bigrams_0, counts_0 = zip(*sorted(top_bigrams_class_0, key=lambda x: x[1]))
bigrams_1, counts_1 = zip(*sorted(top_bigrams_class_1, key=lambda x: x[1]))

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(bg) for bg in bigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Bigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(bg) for bg in bigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Bigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('/kaggle/working/images/top-10-bigrams.png', dpi=300, bbox_inches='tight')
plt.show()

```



Top-10 Trigrams

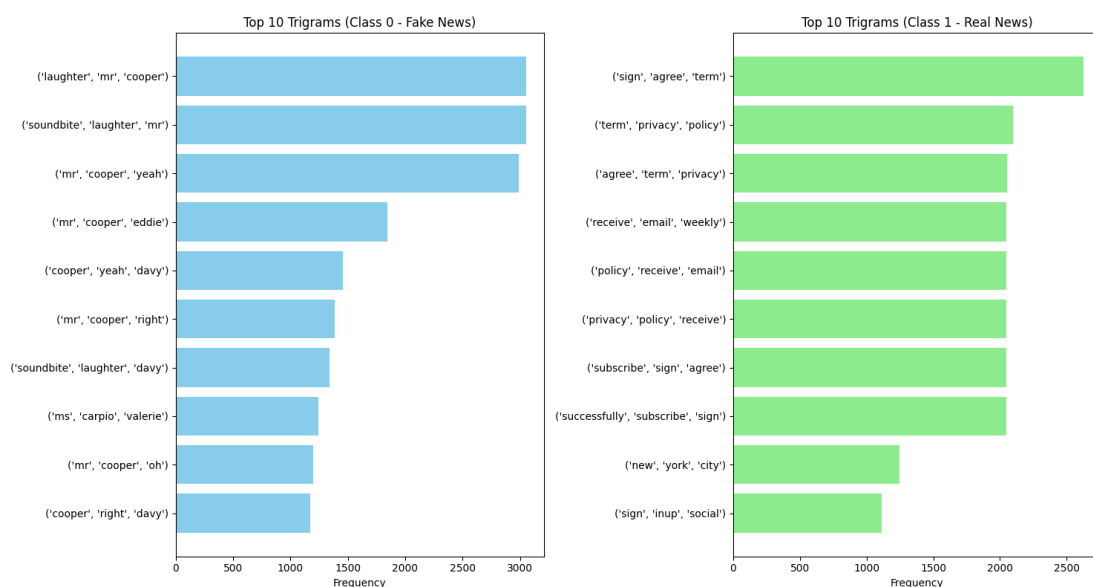
```

In [30]: ▶ # Filter and plot top-10 trigrams for both fake and real news articles
def get_top_trigrams(text_series):
    all_trigrams = []
    for text in text_series.dropna():
        tokens = word_tokenize(text.lower())
        trigram_list = list(trigrams(tokens))
        all_trigrams.extend(trigram_list)
    return Counter(all_trigrams).most_common(10)

# Filter data for class 0 and class 1
class_0_data = data[data['class'] == 0]['text']
class_1_data = data[data['class'] == 1]['text']
top_trigrams_class_0 = get_top_trigrams(class_0_data)
top_trigrams_class_1 = get_top_trigrams(class_1_data)
trigrams_0, counts_0 = zip(*sorted(top_trigrams_class_0, key=lambda x: x[1]))
trigrams_1, counts_1 = zip(*sorted(top_trigrams_class_1, key=lambda x: x[1]))

# Create subplots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 8))
# Plot for class 0
ax1.barh([str(tg) for tg in trigrams_0], counts_0, color='skyblue')
ax1.set_title('Top 10 Trigrams (Class 0 - Fake News)')
ax1.set_xlabel('Frequency')
# Plot for class 1
ax2.barh([str(tg) for tg in trigrams_1], counts_1, color='lightgreen')
ax2.set_title('Top 10 Trigrams (Class 1 - Real News)')
ax2.set_xlabel('Frequency')
plt.tight_layout()
plt.savefig('/kaggle/working/images/top-10-trigrams.png', dpi=300,
plt.show()

```



3. Modelling

```
In [110]: ▶ # Create a copy of data
df = data.copy()

# Define endog and exog
endog = df["class"]
exog = df[['text']]

# First train-test split (70-30)
X_temp, X_test, y_temp, y_test = train_test_split(exog, endog, test_size=0.3, random_state=42)

# Second split temp into train and validation (70-30 of remaining 70%)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.3, random_state=42)
```

```
In [111]: ▶ # Tokenize text feature for Word2Vec
def tokenize_text(text_series):
    return [word_tokenize(text.lower()) for text in text_series.dropna()]
tokenized_spacy_text = tokenize_text(exog['text'])

# Train Word2Vec model on the tokenized text
word2vec_model = Word2Vec(sentences=tokenized_spacy_text, vector_size=100, min_count=1)

# Function to get average Word2Vec vector for a text
def get_word2vec_vector(text, model, vector_size=100):
    words = word_tokenize(text.lower())
    word_vectors = [model.wv[word] for word in words if word in model.wv]
    if len(word_vectors) == 0:
        return np.zeros(vector_size)
    return np.mean(word_vectors, axis=0)

# Vectorize spacy_text using Word2Vec
X_train_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in X_train])
X_val_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in X_val])
X_test_1 = np.array([get_word2vec_vector(text, word2vec_model) for text in X_test])

# Check shapes
print("X_train shape:", X_train_1.shape)
print("X_val shape:", X_val_1.shape)
print("X_test shape:", X_test_1.shape)

X_train shape: (7406, 100)
X_val shape: (3175, 100)
X_test shape: (4535, 100)
```

Objective 2: Build a Baseline Model (Logistic Regression)

- The target variable's classes are imbalanced. Two strategies to address class imbalance are incorporated in the modelling pipelines.
 - SMOTE** is implemented in a pipeline to generate synthetic samples for the minority class.
 - Random Undersampling** is implemented in a pipeline to reduce entities for the majority class.

```
In [116]: ► # Prepare training and validation sets
X_train_logistic = np.vstack((X_train_1, X_val_1))
y_train_logistic = np.hstack((y_train, y_val))

# Define pipeline for fitting model with imbalanced data
lr_model_imbalanced = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_imbalanced = cross_val_score(lr_model_imbalanced, X_train_logistic, y_train_logistic, cv=5)
print("\n[Pipeline 1: Imbalanced]")
print("Unbalanced X_train_logistic shape:", X_train_logistic.shape)
print("CV Scores:", cv_scores_imbalanced)
print("Average recall:", cv_scores_imbalanced.mean())

# Define pipeline for fitting model with oversampled minority class
smote = SMOTE(sampling_strategy='minority', random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_logistic, y_train_logistic)
lr_model_smote = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_smote = cross_val_score(lr_model_smote, X_train_smote, y_train_smote, cv=5)
print("\n[Pipeline 2: SMOTE Oversampling]")
print("SMOTE-balanced X_train_logistic shape:", X_train_smote.shape)
print("CV Scores:", cv_scores_smote)
print("Average recall:", cv_scores_smote.mean())

# Define pipeline for fitting model with undersampled majority class
rus = RandomUnderSampler(sampling_strategy='majority', random_state=42)
X_train_rus, y_train_rus = rus.fit_resample(X_train_logistic, y_train_logistic)
lr_model_rus = LogisticRegression(max_iter=1000, random_state=42)
cv_scores_rus = cross_val_score(lr_model_rus, X_train_rus, y_train_rus, cv=5)
print("\n[Pipeline 3: Random Undersampling]")
print("RUS-balanced X_train_logistic shape:", X_train_rus.shape)
print("CV Scores:", cv_scores_rus)
print("Average recall:", cv_scores_rus.mean())
```

```
[Pipeline 1: Imbalanced]
Unbalanced X_train_logistic shape: (10581, 100)
CV Scores: [0.69278891 0.66515936 0.69772238 0.67549447 0.69182912]
Average recall: 0.6845988454089068
```

```
[Pipeline 2: SMOTE Oversampling]
SMOTE-balanced X_train_logistic shape: (16404, 100)
CV Scores: [0.75739808 0.75648344 0.76104342 0.77110291 0.77042683]
Average recall: 0.7632909365199685
```

```
[Pipeline 3: Random Undersampling]
RUS-balanced X_train_logistic shape: (4758, 100)
CV Scores: [0.75         0.75105042 0.75         0.7434498  0.74550199]
Average recall: 0.7480004422821761
```



```

In [118]: # Predictions by model fitted with imbalanced data
lr_model_imbalanced.fit(X_train_logistic, y_train_logistic)
y_pred_imbalanced = lr_model_imbalanced.predict(X_test_1)
print("\nClassification Report : Logistic Regression fitted on imbalanced data")
print(classification_report(y_test, y_pred_imbalanced, target_names=class_names))
y_proba_imbalanced = lr_model_imbalanced.predict_proba(X_test_1)[:, 1]
fpr_imbalanced, tpr_imbalanced, _ = roc_curve(y_test, y_proba_imbalanced)
roc_auc_imbalanced = auc(fpr_imbalanced, tpr_imbalanced)

# Predictions by model fitted with SMOTE balanced data
lr_model_smote.fit(X_train_smote, y_train_smote)
y_pred_smote_balanced = lr_model_smote.predict(X_test_1)
print("\nClassification Report : Logistic Regression fitted on SMOTE balanced data")
print(classification_report(y_test, y_pred_smote_balanced, target_names=class_names))
y_proba_smote = lr_model_smote.predict_proba(X_test_1)[:, 1]
fpr_smote, tpr_smote, _ = roc_curve(y_test, y_proba_smote)
roc_auc_smote = auc(fpr_smote, tpr_smote)

# Predictions by model fitted with RUS balanced data
lr_model_rus.fit(X_train_rus, y_train_rus)
y_pred_rus_balanced = lr_model_rus.predict(X_test_1) # Fixed bug /
print("\nClassification Report : Logistic Regression fitted on RUS balanced data")
print(classification_report(y_test, y_pred_rus_balanced, target_names=class_names))
y_proba_rus = lr_model_rus.predict_proba(X_test_1)[:, 1]
fpr_rus, tpr_rus, _ = roc_curve(y_test, y_proba_rus)
roc_auc_rus = auc(fpr_rus, tpr_rus)

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
cm1 = confusion_matrix(y_test, y_pred_imbalanced)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm1, display_labels=class_names)
disp1.plot(ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title("Imbalanced Data")
cm2 = confusion_matrix(y_test, y_pred_smote_balanced)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=class_names)
disp2.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title("SMOTE Balanced Data")
cm3 = confusion_matrix(y_test, y_pred_rus_balanced)
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm3, display_labels=class_names)
disp3.plot(ax=axes[2], cmap='Blues', colorbar=False)
axes[2].set_title("RUS Balanced Data")
plt.tight_layout()
plt.savefig('/kaggle/working/images/logistic-regression-confusion-matrices.png')
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
plt.plot(fpr_imbalanced, tpr_imbalanced, label=f"Imbalanced (AUC = {roc_auc_imbalanced:.2f})")
plt.plot(fpr_smote, tpr_smote, label=f"SMOTE Oversampling (AUC = {roc_auc_smote:.2f})")
plt.plot(fpr_rus, tpr_rus, label=f"Random Undersampling (AUC = {roc_auc_rus:.2f})")
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for Logistic Regression with Different Sampling Methods")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.savefig('/kaggle/working/images/logistic-regression-ROC-AUC.png')

```

```
plt.show()
```

Classification Report : Logistic Regression fitted on imbalanced data

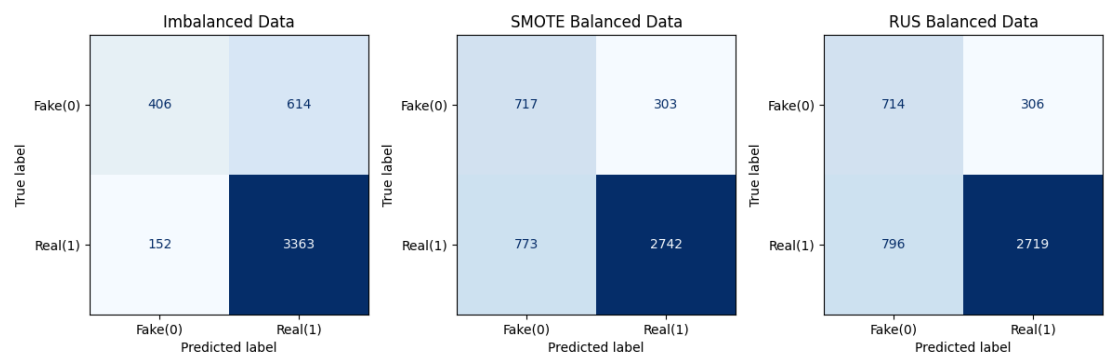
	precision	recall	f1-score	support
Fake(0)	0.7276	0.3980	0.5146	1020
Real (1)	0.8456	0.9568	0.8978	3515
accuracy			0.8311	4535
macro avg	0.7866	0.6774	0.7062	4535
weighted avg	0.8191	0.8311	0.8116	4535

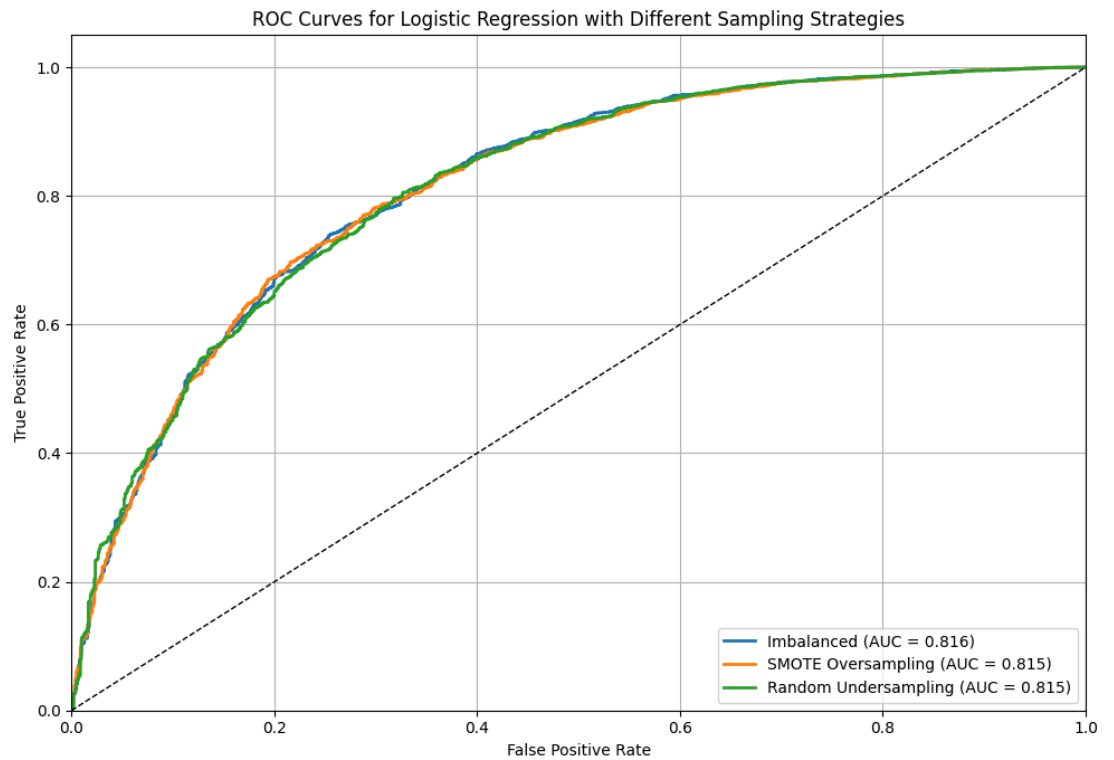
Classification Report : Logistic Regression fitted on SMOTE balanced data

	precision	recall	f1-score	support
Fake(0)	0.4812	0.7029	0.5713	1020
Real (1)	0.9005	0.7801	0.8360	3515
accuracy			0.7627	4535
macro avg	0.6909	0.7415	0.7036	4535
weighted avg	0.8062	0.7627	0.7764	4535

Classification Report : Logistic Regression fitted on RUS balanced data

	precision	recall	f1-score	support
Fake(0)	0.4728	0.7000	0.5644	1020
Real (1)	0.8988	0.7735	0.8315	3515
accuracy			0.7570	4535
macro avg	0.6858	0.7368	0.6980	4535
weighted avg	0.8030	0.7570	0.7714	4535





The baseline model fitted with SMOTE-balanced data yields better performance since the aim is to catch as many fake news articles as possible.

- Its **recall** score for the minority class Fake (0) is highest at 0.7029.
- Its **macro-averaged recall** is highest at 0.7415.

Objective 3: Build an XGBoost model, LSTM neural network, and Finetune the RoBERTa-base Transformer

XGBoost Model

```
In [119]: # Prepare training and validation sets
X_train_xgb = np.vstack((X_train_1, X_val_1))
y_train_xgb = np.hstack((y_train, y_val))

# Define pipeline for fitting model with imbalanced data
xgb_model_imbalanced = XGBClassifier(use_label_encoder=False, eval_
cv_scores_imbalanced = cross_val_score(xgb_model_imbalanced, X_train_xgb, y_train_xgb, cv=5)
print("\n[Pipeline 1: Imbalanced]")
print("Unbalanced X_train_xgb shape:", X_train_xgb.shape)
print("CV Scores:", cv_scores_imbalanced)
print("Average recall:", cv_scores_imbalanced.mean())

# Define pipeline for fitting model with oversampled minority class
smote = SMOTE(sampling_strategy='minority', random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_xgb, y_train_xgb)
xgb_model_smote = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
cv_scores_smote = cross_val_score(xgb_model_smote, X_train_smote, y_train_smote, cv=5)
print("\n[Pipeline 2: SMOTE Oversampling]")
print("SMOTE-balanced X_train_xgb shape:", X_train_smote.shape)
print("CV Scores:", cv_scores_smote)
print("Average recall:", cv_scores_smote.mean())

# Define pipeline for fitting model with undersampled majority class
rus = RandomUnderSampler(sampling_strategy='majority', random_state=42)
X_train_rus, y_train_rus = rus.fit_resample(X_train_xgb, y_train_xgb)
xgb_model_rus = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
cv_scores_rus = cross_val_score(xgb_model_rus, X_train_rus, y_train_rus, cv=5)
print("\n[Pipeline 3: Random Undersampling]")
print("RUS-balanced X_train_xgb shape:", X_train_rus.shape)
print("CV Scores:", cv_scores_rus)
print("Average recall:", cv_scores_rus.mean())
```

```
[Pipeline 1: Imbalanced]
Unbalanced X_train_xgb shape: (10581, 100)
CV Scores: [0.69994149 0.6917273  0.71131123 0.70792171 0.7225222
1]
Average recall: 0.7066847884361707
```

```
[Pipeline 2: SMOTE Oversampling]
SMOTE-balanced X_train_xgb shape: (16404, 100)
CV Scores: [0.86163739 0.87838822 0.91406341 0.92564506 0.9189024
4]
Average recall: 0.8997273004265691
```

```
[Pipeline 3: Random Undersampling]
RUS-balanced X_train_xgb shape: (4758, 100)
CV Scores: [0.75210084 0.75420168 0.75210084 0.74029854 0.7297479
]
Average recall: 0.7456899601946041
```



```

In [120]: # Predictions by model fitted with imbalanced data
xgb_model_imbalanced.fit(X_train_xgb, y_train_xgb)
y_pred_imbalanced = xgb_model_imbalanced.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on imbalanced data")
print(classification_report(y_test, y_pred_imbalanced, target_names=
y_proba_imbalanced = xgb_model_imbalanced.predict_proba(X_test_1)[
fpr_imbalanced, tpr_imbalanced, _ = roc_curve(y_test, y_proba_imbalanced)
roc_auc_imbalanced = auc(fpr_imbalanced, tpr_imbalanced)

# Predictions by model fitted with SMOTE balanced data
xgb_model_smote.fit(X_train_smote, y_train_smote)
y_pred_smote_balanced = xgb_model_smote.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on SMOTE balanced data")
print(classification_report(y_test, y_pred_smote_balanced, target_names=
y_proba_smote = xgb_model_smote.predict_proba(X_test_1)[:, 1]
fpr_smote, tpr_smote, _ = roc_curve(y_test, y_proba_smote)
roc_auc_smote = auc(fpr_smote, tpr_smote)

# Predictions by model fitted with RUS balanced data
xgb_model_rus.fit(X_train_rus, y_train_rus)
y_pred_rus_balanced = xgb_model_rus.predict(X_test_1)
print("\nClassification Report : XGBoost fitted on RUS balanced data")
print(classification_report(y_test, y_pred_rus_balanced, target_names=
y_proba_rus = xgb_model_rus.predict_proba(X_test_1)[:, 1]
fpr_rus, tpr_rus, _ = roc_curve(y_test, y_proba_rus)
roc_auc_rus = auc(fpr_rus, tpr_rus)

# Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
cm1 = confusion_matrix(y_test, y_pred_imbalanced)
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm1, display_labels=
disp1.plot(ax=axes[0], cmap='Blues', colorbar=False)
axes[0].set_title("Imbalanced Data")
cm2 = confusion_matrix(y_test, y_pred_smote_balanced)
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm2, display_labels=
disp2.plot(ax=axes[1], cmap='Blues', colorbar=False)
axes[1].set_title("SMOTE Balanced Data")
cm3 = confusion_matrix(y_test, y_pred_rus_balanced)
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm3, display_labels=
disp3.plot(ax=axes[2], cmap='Blues', colorbar=False)
axes[2].set_title("RUS Balanced Data")
plt.tight_layout()
plt.savefig('/kaggle/working/images/xgboost-confusion-matrices.png')
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
plt.plot(fpr_imbalanced, tpr_imbalanced, label=f"Imbalanced (AUC = {roc_auc_imbalanced})")
plt.plot(fpr_smote, tpr_smote, label=f"SMOTE Oversampling (AUC = {roc_auc_smote})")
plt.plot(fpr_rus, tpr_rus, label=f"Random Undersampling (AUC = {roc_auc_rus})")
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves for XGBoost with Different Sampling Strategies")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.savefig('/kaggle/working/images/xgboost-ROC-AUC.png', dpi=300,

```

```
plt.show()
```

Classification Report : XGBoost fitted on imbalanced data

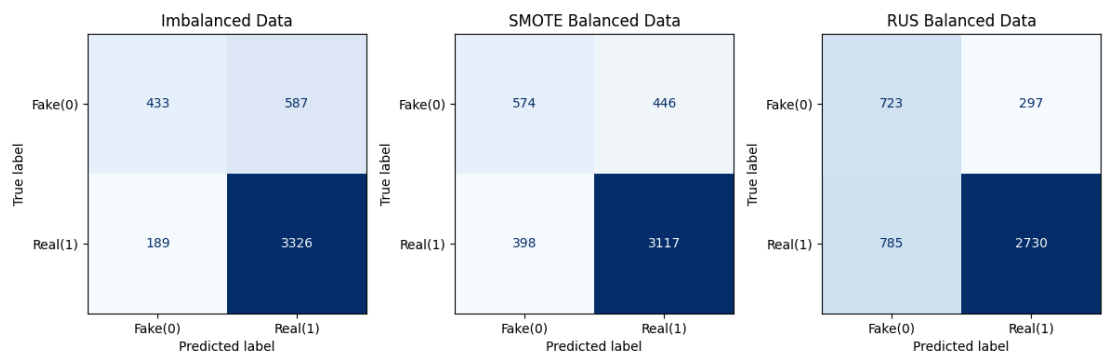
	precision	recall	f1-score	support
Fake(0)	0.6961	0.4245	0.5274	1020
Real (1)	0.8500	0.9462	0.8955	3515
accuracy			0.8289	4535
macro avg	0.7731	0.6854	0.7115	4535
weighted avg	0.8154	0.8289	0.8127	4535

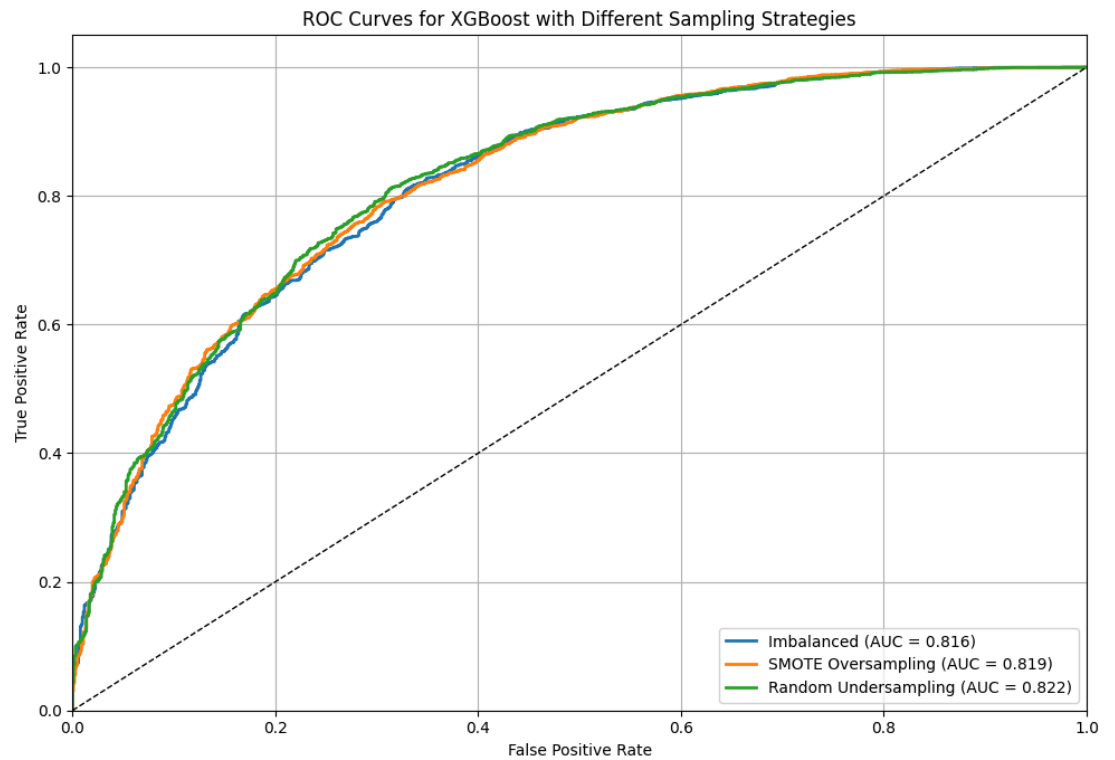
Classification Report : XGBoost fitted on SMOTE balanced data

	precision	recall	f1-score	support
Fake(0)	0.5905	0.5627	0.5763	1020
Real (1)	0.8748	0.8868	0.8808	3515
accuracy			0.8139	4535
macro avg	0.7327	0.7248	0.7285	4535
weighted avg	0.8109	0.8139	0.8123	4535

Classification Report : XGBoost fitted on RUS balanced data

	precision	recall	f1-score	support
Fake(0)	0.4794	0.7088	0.5720	1020
Real (1)	0.9019	0.7767	0.8346	3515
accuracy			0.7614	4535
macro avg	0.6907	0.7427	0.7033	4535
weighted avg	0.8069	0.7614	0.7755	4535





The XGBoost model fitted with RUS-balanced data yields better performance since the aim is to catch as many fake news articles as possible.

- Its **recall** score for the minority class Fake (0) is highest at 0.7088.
- Its **macro-averaged recall** is highest at 0.7427.

LSTM Neural Network

```

In [140]: ▶ # Define Parameters
max_len = 300 # Max tokens per sample
embedding_dim = 100 # As set in Word2Vec

# Define function to convert the text feature to vector sequences
def text_to_vector_sequence(text, model, max_len=300, embedding_dim):
    tokens = word_tokenize(text.lower())
    sequence = [model.wv[token] for token in tokens if token in model.wv]
    if len(sequence) == 0:
        return np.zeros((max_len, embedding_dim))
    sequence = sequence[:max_len] # Truncate
    if len(sequence) < max_len: # Pad
        padding = np.zeros((max_len - len(sequence), embedding_dim))
        sequence = np.vstack((sequence, padding))
    return np.array(sequence)

# Define function to compute f1-score metric
class F1Score(tf.keras.metrics.Metric):
    def __init__(self, name='f1_score', **kwargs):
        super().__init__(name=name, **kwargs)
        self.precision = Precision()
        self.recall = Recall()
    def update_state(self, y_true, y_pred, sample_weight=None):
        y_pred_binary = tf.round(y_pred)
        self.precision.update_state(y_true, y_pred_binary, sample_weight)
        self.recall.update_state(y_true, y_pred_binary, sample_weight)
    def result(self):
        p = self.precision.result()
        r = self.recall.result()
        return 2 * ((p * r) / (p + r + tf.keras.backend.epsilon()))
    def reset_state(self):
        self.precision.reset_state()
        self.recall.reset_state()

# Define function build the LSTM models' architecture
def build_lstm_model(max_len, embedding_dim):
    inputs = Input(shape=(max_len, embedding_dim))
    x = Masking(mask_value=0.0)(inputs)
    x = LSTM(64)(x)
    x = Dropout(0.3)(x)
    x = Dense(32, activation='relu')(x)
    outputs = Dense(1, activation='sigmoid')(x)
    model = Model(inputs, outputs)
    return model

```



```

In [141]: # Define function to train and evaluate the models' respective per
def train_and_evaluate_pipeline(
    X_train_text, X_val_text, X_test_text,
    y_train, y_val, y_test,
    word2vec_model, max_len, embedding_dim,
    sampling_strategy=None, name="Imbalanced"):
    print(f"\n Training on {name} dataset")

    # Convert the text feature to sequences
    X_train_seq = np.array([text_to_vector_sequence(text, word2vec_
    X_val_seq = np.array([text_to_vector_sequence(text, word2vec_mo
    X_test_seq = np.array([text_to_vector_sequence(text, word2vec_r

    # Reshape for resampling (flatten to 2D)
    if sampling_strategy:
        X_train_flat = X_train_seq.reshape((X_train_seq.shape[0], -1))
        X_val_flat = X_val_seq.reshape((X_val_seq.shape[0], -1))
        if sampling_strategy == "smote":
            sampler = SMOTE(sampling_strategy='minority', random_st
        elif sampling_strategy == "rus":
            sampler = RandomUnderSampler(sampling_strategy='majorit
        else:
            raise ValueError("sampling_strategy must be 'smote' or
    X_train_resampled, y_train_resampled = sampler.fit_resample
    X_val_resampled, y_val_resampled = sampler.fit_resample(X_v
    # Reshape back to 3D
    X_train_seq = X_train_resampled.reshape((-1, max_len, embed
    X_val_seq = X_val_resampled.reshape((-1, max_len, embedding
    y_train_used, y_val_used = y_train_resampled, y_val_resamp

    else:
        y_train_used, y_val_used = y_train, y_val

    # Build and compile model
    model = build_lstm_model(max_len, embedding_dim)
    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=[
            Precision(name='precision'),
            Recall(name='recall'),
            F1Score(name='f1_score')])

    # Train
    history = model.fit(
        X_train_seq, y_train_used,
        validation_data=(X_val_seq, y_val_used),
        epochs=6,
        batch_size=64,
        verbose=1)

    # Extract metrics per epoch
    metrics_df = pd.DataFrame({
        'epoch': range(1, len(history.history['loss']) + 1),
        'train_loss': history.history['loss'],
        'val_loss': history.history['val_loss'],
        'val_precision': history.history['val_precision'],
        'val_recall': history.history['val_recall'],
        'val_f1_score': history.history['f1_score']})
    print(f"\n Metrics per epoch - {name}:")
    print(metrics_df.round(4))

```

```
# Plot Train vs Val Loss plots
plt.figure(figsize=(8, 5))
plt.plot(metrics_df['epoch'], metrics_df['train_loss'], label='Train Loss')
plt.plot(metrics_df['epoch'], metrics_df['val_loss'], label='Val Loss')
plt.title(f'Training vs Validation Loss - {name}')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Save training loss vs. validation loss plots to /kaggle/work
save_name = name.replace(" ", "_").replace("-", "_")
plot_path = os.path.join(images_dir, f"LSTM-loss-curve-({save_name})")
plt.savefig(plot_path)
plt.show()

# Test set prediction
y_test_pred_proba = model.predict(X_test_seq).ravel()
y_test_pred = (y_test_pred_proba > 0.5).astype(int)
return history, metrics_df, model, y_test_pred_proba, y_test_pred
```

```

In [142]: ▶ # Run pipelines
           results = {}

           # Imbalanced
           print("="*60)
           history_imb, df_imb, model_imb, y_test_proba_imb, y_test_pred_imb,
               X_train['text'], X_val['text'], X_test['text'],
               y_train, y_val, y_test,
               word2vec_model, max_len, embedding_dim,
               sampling_strategy=None,
               name="Imbalanced")
           results['Imbalanced'] = (y_test_true, y_test_pred_imb, y_test_proba_imb)

           # SMOTE
           print("="*60)
           history_smote, df_smote, model_smote, y_test_proba_smote, y_test_pred_smote,
               X_train['text'], X_val['text'], X_test['text'],
               y_train, y_val, y_test,
               word2vec_model, max_len, embedding_dim,
               sampling_strategy="smote",
               name="SMOTE Balanced")
           results['SMOTE'] = (y_test_true, y_test_pred_smote, y_test_proba_smote)

           # RUS
           print("="*60)
           history_rus, df_rus, model_rus, y_test_proba_rus, y_test_pred_rus,
               X_train['text'], X_val['text'], X_test['text'],
               y_train, y_val, y_test,
               word2vec_model, max_len, embedding_dim,
               sampling_strategy="rus",
               name="RUS Balanced")
           results['RUS'] = (y_test_true, y_test_pred_rus, y_test_proba_rus)

```

```

=====

Training on Imbalanced dataset
Epoch 1/6
116/116 ————— 8s 38ms/step - f1_score: 0.8754 - loss: 0.5238 - precision: 0.7849 - recall: 0.9897 - val_f1_score: 0.8852 - val_loss: 0.4785 - val_precision: 0.8014 - val_recall: 0.9886
Epoch 2/6
116/116 ————— 2s 19ms/step - f1_score: 0.8855 - loss: 0.4593 - precision: 0.8070 - recall: 0.9810 - val_f1_score: 0.8801 - val_loss: 0.4558 - val_precision: 0.8231 - val_recall: 0.9456
Epoch 3/6
116/116 ————— 2s 20ms/step - f1_score: 0.8928 - loss: 0.4182 - precision: 0.8312 - recall: 0.9644 - val_f1_score: 0.8881 - val_loss: 0.4397 - val_precision: 0.8196 - val_recall: 0.9691
Epoch 4/6
116/116 ————— 2s 20ms/step - f1_score: 0.8928 - loss: 0.4182 - precision: 0.8312 - recall: 0.9644 - val_f1_score: 0.8881 - val_loss: 0.4397 - val_precision: 0.8196 - val_recall: 0.9691

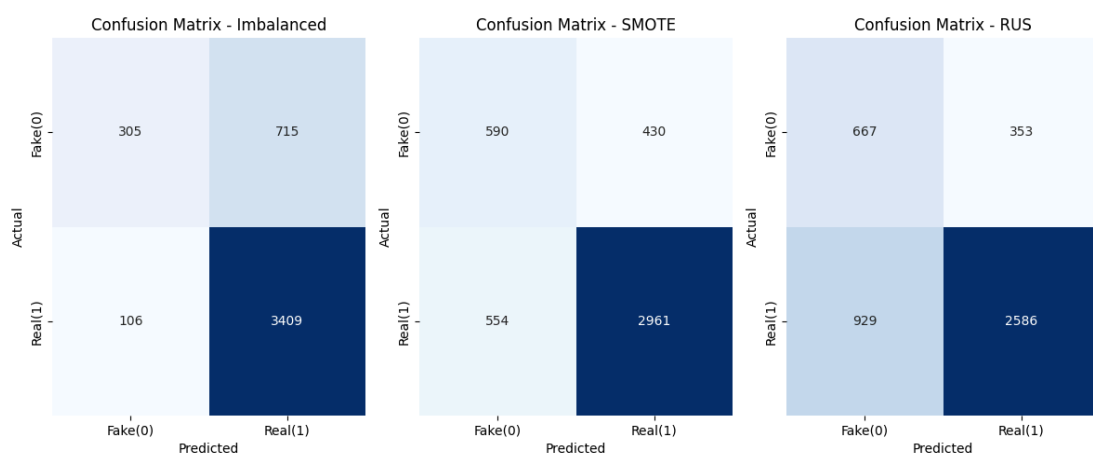
```

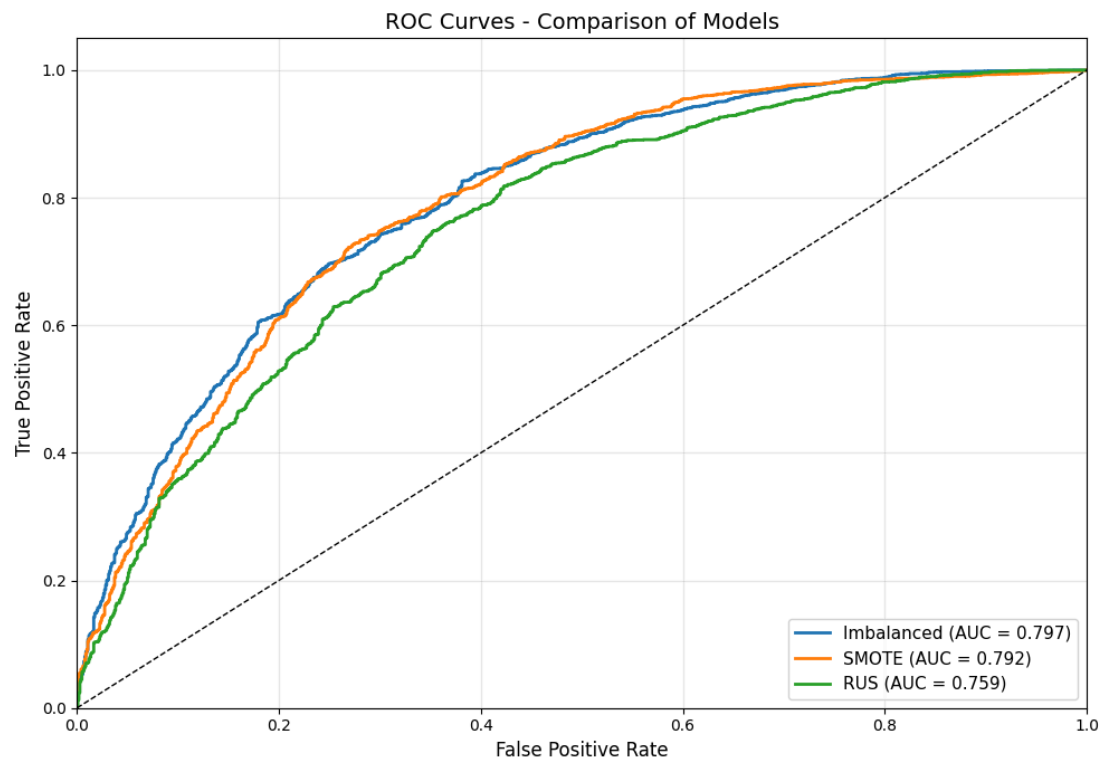
```

In [143]: ▶ # Plot confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
class_names = ['Fake(0)', 'Real(1)']
datasets = ['Imbalanced-LSTM', 'SMOTE-LSTM', 'RUS-LSTM']
for ax, (name, (y_true, y_pred, _)) in zip(axes, results.items()):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        ax=ax,
        cbar=False,
        xticklabels=class_names,
        yticklabels=class_names)
    ax.set_title(f'Confusion Matrix - {name}', fontsize=12)
    ax.set_xlabel('Predicted', fontsize=10)
    ax.set_ylabel('Actual', fontsize=10)
plt.tight_layout()
plt.savefig('/kaggle/working/images/LSTM-confusion_matrices.png', c
plt.show()

# Plot ROC-AUC curves
plt.figure(figsize=(10, 7))
for name, (y_true, _, y_proba) in results.items():
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves - Comparison of Models', fontsize=14)
plt.legend(loc="lower right", fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/images/LSTM-ROC-AUC.png', dpi=300, bbo
plt.show()

```





```
In [144]: ▶ # Print per-class and macro/weighted averages
for name, (y_true, y_pred, y_proba) in results.items():
    print(f"\n {name} Model:")
    print("-" * 40)
    print(classification_report(y_true, y_pred, target_names=['Fake
```

Imbalanced Model:

	precision	recall	f1-score	support
Fake(0)	0.7421	0.2990	0.4263	1020
Real(1)	0.8266	0.9698	0.8925	3515
accuracy			0.8190	4535
macro avg	0.7844	0.6344	0.6594	4535
weighted avg	0.8076	0.8190	0.7877	4535

SMOTE Model:

	precision	recall	f1-score	support
Fake(0)	0.5157	0.5784	0.5453	1020
Real(1)	0.8732	0.8424	0.8575	3515
accuracy			0.7830	4535
macro avg	0.6945	0.7104	0.7014	4535
weighted avg	0.7928	0.7830	0.7873	4535

RUS Model:

	precision	recall	f1-score	support
Fake(0)	0.4179	0.6539	0.5099	1020
Real(1)	0.8799	0.7357	0.8014	3515
accuracy			0.7173	4535
macro avg	0.6489	0.6948	0.6557	4535
weighted avg	0.7760	0.7173	0.7358	4535

The LSTM model fitted with SMOTE-balanced data yields better performance since the aim is to catch as many fake news articles as possible. Its **macro-averaged recall** is highest at 0.7104.

Finetune the RoBERTa-base Transformer

```
In [130]: ▶ # Set up device, pretrained transformer, and tokenizer

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model_name = "roberta-base" # Set up pretrained transformer
roberta_tokenizer = RobertaTokenizer.from_pretrained(model_name) #
Using device: cuda
```

```

In [131]: # Initialize the NewsDataset with text chunking using a sliding window
class NewsDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length=512, stride=128):
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.stride = stride
        self.all_chunk_encodings = [] # Store {'input_ids', 'attention_mask'}
        self.all_chunk_labels = [] # Store the label for each chunk
        self.chunk_original_indices = [] # Store the index of the original text

        # Process each original text to create chunks
        for original_idx, text in enumerate(texts):
            tokenized_output = tokenizer(
                text,
                truncation=False,
                padding=False,
                max_length=self.max_length,
                stride=self.stride,
                return_overflowing_tokens=True,
                return_tensors="pt") # Return PyTorch tensors

            # Iterate through each chunk generated for the current text
            for i in range(len(tokenized_output['input_ids'])):
                chunk_ids = tokenized_output['input_ids'][i]
                chunk_mask = tokenized_output['attention_mask'][i]

                if chunk_ids.shape[0] > max_length: # Truncate
                    chunk_ids = chunk_ids[:max_length]
                    chunk_mask = chunk_mask[:max_length]
                elif chunk_ids.shape[0] < max_length: # Pad input
                    pad_len = max_length - chunk_ids.shape[0]
                    pad_tensor = torch.full((pad_len,), tokenizer.pad_token_id)
                    mask_pad = torch.zeros(pad_len, dtype=torch.long)
                    chunk_ids = torch.cat([chunk_ids, pad_tensor])
                    chunk_mask = torch.cat([chunk_mask, mask_pad])

                self.all_chunk_encodings.append({
                    'input_ids': chunk_ids,
                    'attention_mask': chunk_mask})
                self.all_chunk_labels.append(labels[original_idx])
                self.chunk_original_indices.append(original_idx)

        # Return the total number of chunks across all original articles
        def __len__(self):
            return len(self.all_chunk_labels)

        # Retrieve a single chunk's data and its associated original label
        def __getitem__(self, idx):
            item = {key: val.clone() for key, val in self.all_chunk_encodings[idx]}
            item['labels'] = torch.tensor(self.all_chunk_labels[idx], dtype=torch.long)
            item['original_index'] = torch.tensor(self.chunk_original_indices[idx], dtype=torch.long)
            return item

```



```

In [132]: # Define function to resample imbalanced data before chunking

def resample_data(X_texts, y_labels, strategy="smote"):
    X_list = X_texts.tolist()
    y_array = np.array(y_labels)

    if strategy == "smote": # Oversample the minority class

        # Use dummy features (lengths) to simulate feature space
        X_dummy = np.array([[len(text)] for text in X_list])
        sampler = SMOTE(sampling_strategy='minority', random_state=
        _, y_resampled = sampler.fit_resample(X_dummy, y_array)

        # Resample texts by oversampling the minority class
        from collections import Counter
        counter = Counter(y_array)
        target_count = max(counter.values())
        X_resampled, y_resampled = [], []
        for label in [0, 1]:
            indices = np.where(y_array == label)[0]
            resampled_indices = np.random.choice(indices, size=target_count)
            X_resampled.extend([X_list[i] for i in resampled_indices])
            y_resampled.extend([y_array[i] for i in resampled_indices])

    elif strategy == "rus": # Undersample the majority class
        X_dummy = np.array([[len(text)] for text in X_list])
        sampler = RandomUnderSampler(sampling_strategy='majority',
        _, y_resampled = sampler.fit_resample(X_dummy, y_array)

        # Resample texts by undersampling the majority class according to target count
        from collections import Counter
        counter = Counter(y_array)
        target_count = min(counter.values())
        X_resampled, y_resampled = [], []
        for label in [0, 1]:
            indices = np.where(y_array == label)[0]
            resampled_indices = np.random.choice(indices, size=target_count)
            X_resampled.extend([X_list[i] for i in resampled_indices])
            y_resampled.extend([y_array[i] for i in resampled_indices])

    else: # imbalanced dataset
        X_resampled, y_resampled = X_list, y_array

    return X_resampled, y_resampled

```



```

In [133]: # Initialize the MetricsTracker class
class MetricsTracker:
    def __init__(self):
        self.epoch_data = []
    def log_epoch(self, epoch, metrics):
        self.epoch_data.append({'epoch': int(epoch), **metrics})
    def get_dataframe(self):
        return pd.DataFrame(self.epoch_data).set_index('epoch')
metrics_tracker = MetricsTracker()

# Define function to compute performance metrics
def compute_metrics(p):
    preds = np.argmax(p.predictions, axis=1)
    probs = torch.softmax(torch.tensor(p.predictions), dim=1).numpy()
    return {
        'accuracy': accuracy_score(p.label_ids, preds),
        'precision': precision_score(p.label_ids, preds, zero_divisor=1),
        'recall': recall_score(p.label_ids, preds, zero_division=0),
        'f1': f1_score(p.label_ids, preds, zero_division=0),
        'roc_auc': roc_auc_score(p.label_ids, probs)}

# Initialize the CustomTrainer class for logging
class CustomTrainer(Trainer):
    def evaluate(self, *args, **kwargs):
        metrics = super().evaluate(*args, **kwargs)
        self.control = self.callback_handler.on_evaluate(self.args, metrics)
        metrics_tracker.log_epoch(self.state.epoch, metrics)
        return metrics

# Define function to extract average training loss per epoch from trainer
def extract_per_epoch_train_loss(trainer):
    train_loss_epochs = {}
    for log in trainer.state.log_history:
        if 'loss' in log and 'epoch' in log:
            epoch = round(log['epoch']) # Round float epoch to nearest int
            if epoch not in train_loss_epochs:
                train_loss_epochs[epoch] = []
            train_loss_epochs[epoch].append(log['loss'])
    # Return average loss per epoch
    return pd.Series({epoch: np.mean(losses) for epoch, losses in train_loss_epochs.items()})

# Define training arguments function
def get_training_args(run_name):
    return TrainingArguments(
        output_dir=f"./roberta_results_{run_name}",
        run_name=f"roberta-{run_name}",
        num_train_epochs=4,
        per_device_train_batch_size=16,
        per_device_eval_batch_size=32,
        learning_rate=2e-5,
        warmup_ratio=0.06,
        weight_decay=0.01,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="f1",
        greater_is_better=True,
        logging_dir=f"./roberta_logs_{run_name}",
        logging_steps=100,
        seed=42,
        fp16=torch.cuda.is_available(),

```

```
report_to="none",  
save_total_limit=2)
```


In [134]: **# Define function to implement finetuning pipelines**

```
def finetune_roberta_pipeline(X_train, y_train, X_val, y_val, X_test, y_test, name):
    print(f"\n Fine-tuning RoBERTa ({name})")
    X_train_res, y_train_res = resample_data(X_train, y_train, stratify=y_train)

    # Create chunked datasets
    train_dataset = NewsDataset(X_train_res, y_train_res, roberta_tokenizer)
    val_dataset = NewsDataset(X_val.tolist(), y_val.tolist(), roberta_tokenizer)
    test_dataset = NewsDataset(X_test.tolist(), y_test.tolist(), roberta_tokenizer)

    # Load model
    model = RobertaForSequenceClassification.from_pretrained(
        model_name,
        num_labels=2,
        hidden_dropout_prob=0.2,
        attention_probs_dropout_prob=0.2
    ).to(device)

    # Reset tracker and trainer
    global metrics_tracker
    metrics_tracker = MetricsTracker()
    trainer_args = get_training_args(name.replace(" ", "_"))
    trainer = CustomTrainer(
        model=model,
        args=trainer_args,
        train_dataset=train_dataset,
        eval_dataset=val_dataset,
        compute_metrics=compute_metrics,)

    # Train and compile the performance metrics DataFrame
    trainer.train()
    avg_train_loss = extract_per_epoch_train_loss(trainer)
    metrics_df = metrics_tracker.get_dataframe()
    metrics_df = metrics_df.assign(
        train_loss=metrics_df.index.map(avg_train_loss), # Map and
        val_loss=metrics_df['eval_loss'])
    print(f"\nMetrics per epoch - {name}:")
    print(metrics_df[['train_loss', 'val_loss', 'eval_precision',

    # Plot and Save Loss Curve
    plt.figure(figsize=(8, 5))
    epochs = metrics_df.index
    plt.plot(epochs, metrics_df['train_loss'], label='Train Loss',
    plt.plot(epochs, metrics_df['val_loss'], label='Validation Loss')
    plt.title(f'Training vs Validation Loss - RoBERTa ({name})')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plot_path = f"/kaggle/working/images/roberta_loss_{name.replace(' ', '_')}.png"
    plt.savefig(plot_path)
    plt.show()

    # Evaluate respective performance on the test set
    test_outs = trainer.predict(test_dataset)
    y_test_pred = np.argmax(test_outs.predictions, axis=1)
    y_test_proba = torch.softmax(torch.tensor(test_outs.predictions), dim=-1)

    return metrics_df, model, y_test_pred, y_test_proba, y_test
```

```
In [135]: # Run Pipelines
results = {}

# Imbalanced
print("="*60)
df_imb, model_imb, y_test_pred_imb, y_test_proba_imb, y_test_true = finetune_
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy=None, name="Imbalanced")
results['RoBERTa-Imbalanced'] = (y_test_true, y_test_pred_imb, y_test_proba_imb)

# SMOTE
print("="*60)
df_smote, model_smote, y_test_pred_smote, y_test_proba_smote, _ = finetune_
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy="smote", name="SMOTE")
results['RoBERTa-SMOTE'] = (y_test_true, y_test_pred_smote, y_test_proba_smote)

# RUS
print("="*60)
df_rus, model_rus, y_test_pred_rus, y_test_proba_rus, _ = finetune_
    X_train['text'], y_train, X_val['text'], y_val, X_test['text'],
    strategy="rus", name="RUS")
results['RoBERTa-RUS'] = (y_test_true, y_test_pred_rus, y_test_proba_rus)
```

=====

Fine-tuning RoBERTa (Imbalanced)

Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at roberta-base and are newly initialized: ['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

[928/928 31:32, Epoch 4/4]

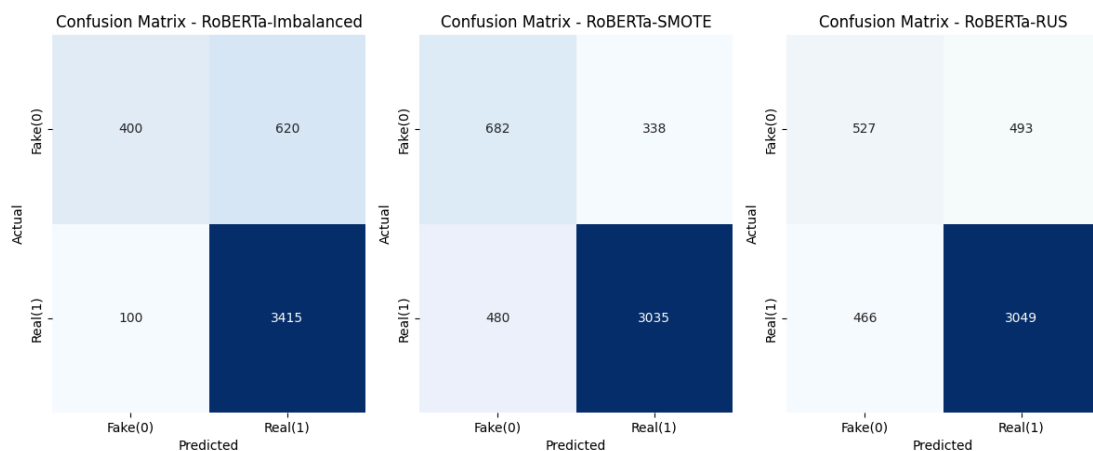
Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1	Roc Auc
1	0.488200	0.414061	0.824567	0.867284	0.913450	0.889768	0.813613
2	0.404200	0.404004	0.848819	0.849876	0.977651	0.909297	0.839643
3	0.350900	0.412760	0.833071	0.892320	0.892320	0.892320	0.846552

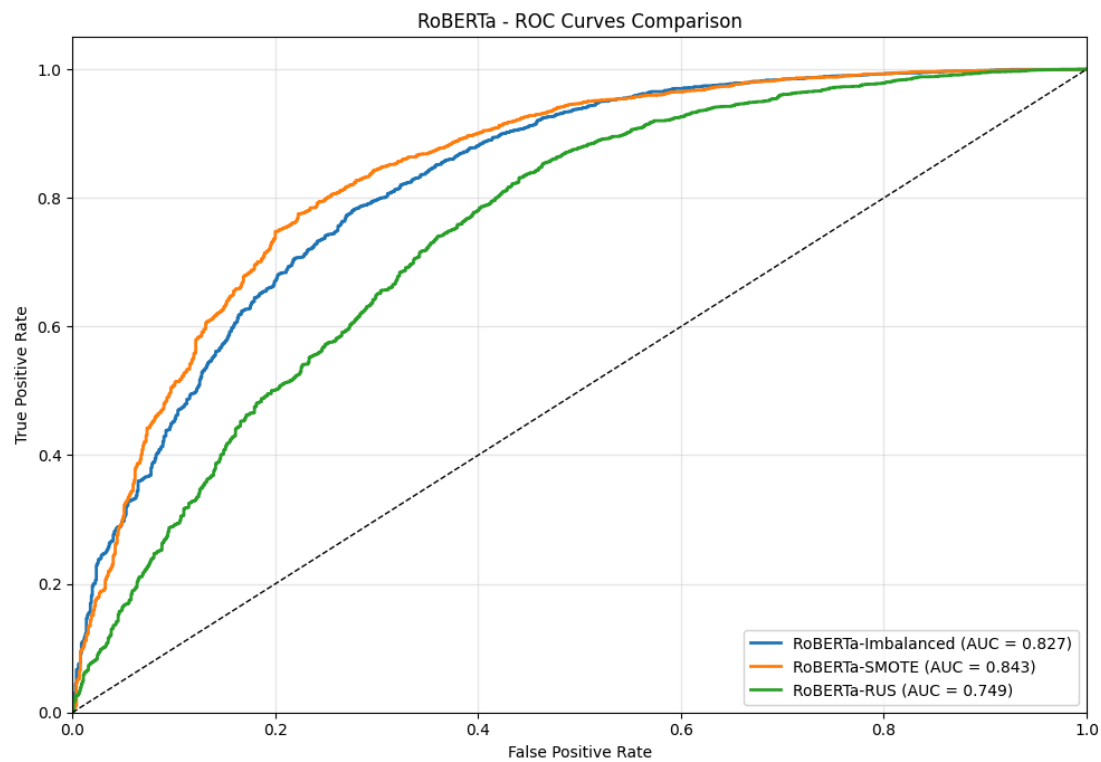
```

In [136]: ▶ # Plot Confusion Matrices
fig, axes = plt.subplots(1, 3, figsize=(12, 5))
class_names = ['Fake(0)', 'Real(1)']
for ax, (name, (y_true, y_pred, _)) in zip(axes, results.items()):
    cm = confusion_matrix(y_true, y_pred)
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        ax=ax,
        cbar=False,
        xticklabels=class_names,
        yticklabels=class_names)
    ax.set_title(f'Confusion Matrix - {name}', fontsize=12)
    ax.set_xlabel('Predicted', fontsize=10)
    ax.set_ylabel('Actual', fontsize=10)
plt.tight_layout()
plt.savefig('/kaggle/working/images/roberta-confusion_matrices.png')
plt.show()

# Plot ROC-AUC curve plots
plt.figure(figsize=(10, 7))
for name, (y_true, _, y_proba) in results.items():
    fpr, tpr, _ = roc_curve(y_true, y_proba)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, lw=2, label=f'{name} (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], 'k--', linewidth=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('RoBERTa - ROC Curves Comparison')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('/kaggle/working/images/roberta-roc-auc.png', dpi=300,
plt.show()

```





```
In [138]: # Print per-class and macro/weighted averages
for name, (y_true, y_pred, y_proba) in results.items():
    print(f"\n {name} Model:")
    print("-" * 40)
    print(classification_report(y_true, y_pred, target_names=['Fake
```

RoBERTa-Imbalanced Model:

	precision	recall	f1-score	support
Fake(0)	0.8000	0.3922	0.5263	1020
Real(1)	0.8463	0.9716	0.9046	3515
accuracy			0.8412	4535
macro avg	0.8232	0.6819	0.7155	4535
weighted avg	0.8359	0.8412	0.8195	4535

RoBERTa-SMOTE Model:

	precision	recall	f1-score	support
Fake(0)	0.5869	0.6686	0.6251	1020
Real(1)	0.8998	0.8634	0.8812	3515
accuracy			0.8196	4535
macro avg	0.7434	0.7660	0.7532	4535
weighted avg	0.8294	0.8196	0.8236	4535

RoBERTa-RUS Model:

	precision	recall	f1-score	support
Fake(0)	0.5307	0.5167	0.5236	1020
Real(1)	0.8608	0.8674	0.8641	3515
accuracy			0.7885	4535
macro avg	0.6958	0.6920	0.6939	4535
weighted avg	0.7866	0.7885	0.7875	4535

The findings justify that balancing the dataset used for finetuning pretrained transformers improves performance on the minority class.

- The RoBERTa transformer finetuned with SMOTE-balanced data **RoBERTa-SMOTE** scores a recall of **0.6686** for the minority class Fake(0) and achieves the highest macro-averaged recall (**0.7660**) indicating that the model performs best in terms of capturing both classes equally well.
- The RoBERTa transformer finetuned with RUS-balanced data **RoBERTa-RUS** scores a recall of **0.5167** for the minority class Fake(0) and a macro-averaged recall of **0.6920**.
- The RoBERTa transformer finetuned with imbalanced data **RoBERTa-Imbalanced** achieves the lowest recall of **0.3922** and a macro-averaged recall (**0.6819**).

```
In [152]: # # Save Fine-tuned Models
# import os
# import shutil

# # Base directory to save models
# models_save_dir = "/kaggle/working/roberta_models_tuned"
# os.makedirs(models_save_dir, exist_ok=True)
# models_to_save = {
#     "roberta_imbalanced": model_imb,
#     "roberta_smote": model_smote,
#     "roberta_rus": model_rus}

# print("Saving fine-tuned RoBERTa models")

# for model_name, model in models_to_save.items():
#     model_path = os.path.join(models_save_dir, model_name)

#     # Remove old directory if exists
#     if os.path.exists(model_path):
#         shutil.rmtree(model_path)

#     # Save model and tokenizer
#     model.save_pretrained(model_path)
#     roberta_tokenizer.save_pretrained(model_path)
#     print(f"Model saved to: {model_path}")
```

4. Evaluation and Model Interpretability

Objective 4: Interpret best performing ML model using the LIME library

Best Performing Model

MODEL	Macro-averaged Recall	Recall (Fake)	Recall (Real)	Precision (Fake)	Precision (Real)	Macro-averaged f1	Accuracy	AUC
Logistic Regression-Imbalanced	0.6774	0.3980	0.9568	0.7276	0.8456	0.7062	0.8311	0.816
Logistic Regression-SMOTE	0.7145	0.7029	0.7801	0.4812	0.9005	0.7036	0.7627	0.815
Logistic Regression-RUS	0.7368	0.7000	0.7735	0.4728	0.8988	0.6980	0.7570	0.815
XGBoost-Imbalanced	0.6854	0.4245	0.9462	0.6961	0.8500	0.7115	0.8289	0.816
XGBoost-SMOTE	0.7248	0.5627	0.8868	0.5909	0.8748	0.7285	0.8139	0.819
XGBoost-RUS	0.7427	0.7088	0.7767	0.4794	0.9019	0.7033	0.7614	0.822
LSTM-Imbalanced	0.6344	0.2990	0.9698	0.7421	0.8266	0.6594	0.8190	0.797
LSTM-SMOTE	0.7104	0.5784	0.8424	0.5157	0.8732	0.7014	0.7830	0.792

MODEL	Macro-averaged Recall	Recall (Fake)	Recall (Real)	Precision (Fake)	Precision (Real)	Macro-averaged f1	Accuracy	AUC
LSTM-RUS	0.6948	0.6539	0.7357	0.4179	0.8799	0.6557	0.7173	0.759
RoBERTa-Imbalanced	0.6819	0.3922	0.9316	0.8000	0.8463	0.7155	0.8412	0.827
RoBERTa-SMOTE	0.7660	0.6686	0.8634	0.5869	0.8998	0.7532	0.8196	0.843

The Finetuned RoBERTa model using SMOTE-balanced dataset is chosen as the better performing model because it achieves the highest scores for **Macro-averaged recall** (0.7660). This is corroborated by the confusion matrix, which reports the highest true positives for accurate fake news predictions (682). The ROC-AUC curve plots reinforces this decision since with the finetuned transformer with SMOTE-balanced data achieves the highest AUC (0.843). RoBERTa's contextual understanding from pretraining and fine-tuning with balanced data enhances its robustness, and generalizability.

Interpret the Finetuned RoBERTa-base Model

We utilize the LIME (Local Interpretable Model-agnostic Explanations) library to interpret the predictions of a fine-tuned RoBERTa model, which classifies news articles as either class 0 (fake) or class 1 (real). We load the pre-trained RoBERTa model and its tokenizer from a specified path, setting the model to evaluation mode to disable gradient computation. A predictor function tokenizes input texts, processes them through the model, and applies a softmax function to convert logits into probability scores for the two classes.

The LIME explainer is initialized with class names 'real' and 'fake', enabling local interpretation of a selected news article (e.g., `data.iloc[99, 10]`). It generates explanations by perturbing the text and evaluating the model's predictions on these variations, using 500 samples and highlighting the top 10 features. The results are visualized as a 1x2 horizontal bar plot, with one subplot per class. Bars represent feature weights:

- Green for positive influence (supporting the class).
- Red for negative influence.

The bars are sorted by absolute weight to emphasize the most impactful words. Leveraging the LIME library provides insight into which terms drive the model's classification, enhancing interpretability of the fake/real news prediction.

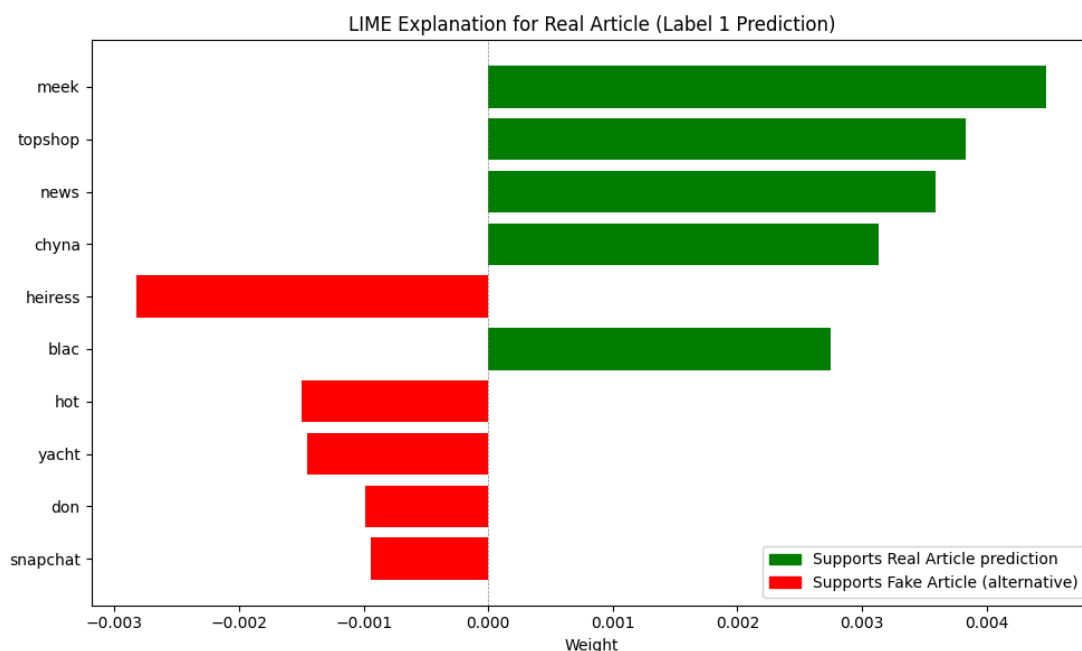
```
In [150]: ▶ # Preview a random sample
data.iloc[99, 10]
```

```
Out[150]: 'blac chyna cozy hot felon jeremy meek pic look like blac chyna h
ot felon jeremy meek work chyna share picture posing arm snapchat
wednesday night show tattoo chyna don skintight lace orange dress
meek keep casual camouflage print shirt rip jean news early snapc
hat post chyna appear photo shoot lead speculate possibly star ca
mpaign meek model attractive mugshot go viral headline summer rel
ationship topshop heiress chloe green meek photograph kiss green
luxury yacht coast turkey july married wife melissa time shortly
picture surface meek file separation melissa year marriage news m
eek green shy pack pda watch video'
```



```
plt.tight_layout()
plt.savefig('./images/lime_explanation_barplot.png', dpi=300, bbox_
plt.show()
```

Predicted Class: Real Article (Label 1)



5. Deployment

Objective 5: Deploy selected model using FastAPI and Streamlit

Thresholding Strategy

An asymmetric thresholding strategy was adopted to further improve the model's **recall** at the expense of a slight drop in **precision** as follows:

- The threshold for predicting Fake Articles was lowered to **0.45**.
- The threshold for predicting Real Articles was raised to **0.60**.
- Probabilities between (**0.40 to 0.45**) for **Label 0** and (**0.55 to 0.60**) for **Label 1** were flagged for human review.

Containerization Strategy

We leveraged FASTAPI, Streamlit, and Docker to deploy the fine-tuned RoBERTa Transformer Model for fake news detection.

- **Back-end** : The FastAPI app (**main.py**) is responsible for model inference. It loads the saved roberta_model, incorporates the SpaCy text preprocessing pipeline, scrapes article text from a provided URL using BeautifulSoup, predicts whether it is fake or real, and generates LIME explanation results.
- **Front-end** : The Streamlit app (**app.py**) provides an interactive frontend web interface that prompts the user to enter a news article URL for prediction. It sends the URL to the FastAPI endpoint (/predict/) via localhost, displays the article title, prediction label, and generates a LIME explanation plot.

- **Docker** : Used to containerize the front-end and the back-end using two Dockerfiles (one for each).
 - The required Python dependencies for the front-end include: streamlit, requests, and matplotlib.
 - The required Python dependencies for the back-end include: fastapi, uvicorn, transformers, torch, numpy, lime, pydantic, requests, beautifulsoup4, spacy, and lxml.
 - The docker-compose.yml exposes ports: **8000:8000** to the back-end and ports

6. Conclusion, Recommendations, and Next Steps

6.1 Conclusion

This project successfully developed a robust pipeline for fake news detection. It leveraged advanced NLP techniques and multiple machine learning approaches. The dataset (15,116 news articles) was meticulously processed through feature engineering (domain extraction, text length analysis, preprocessing) and exploratory analysis. The **macro-averaged recall** metric was selected as the success criteria because the project aims to catch fake news articles but the training dataset is imbalanced. The Key objectives were addressed by:

- Utilizing the spaCy library to preprocess text data and perform EDA to unveil underlying characteristics among fake and real news articles.
- Building a baseline **Logistic Regression** model to establish initial performance benchmarks.
- Building an ensemble model (**XGBoost**), a neural network (**LSTM**), and finetuning the **RoBERTa-base** transformer.
- Evaluating the models' respective performance on test set and interpreting the alternative with the highest **macro-averaged recall** using LIME. We successfully met the goal we had set when defining the project's success criteria (the **Finetuned RoBERTa-base transformer with SMOTE-balanced dataset** achieved a macro-averaged recall score of over 76%).
- Deploying the selected ML model using FastAPI, and Streamlit. The deployed model can be integrated with social media apps and browser extensions for real-time fake news detection,

6.2 Recommendations

- 
- **Collaborate with Media Outlets** : Work with news organizations and social media platforms to incorporate this model into their systems for real-time fake news detection.
 - **Educate the Public** : Raise awareness about fake news detection tools, and encourage people to use these technologies for more informed decision-making.
 - **Expand into Multiple Languages** : Ensure the model can detect fake news in various languages to reach a broader audience globally.

6.3 Next Steps

1. Deploy the containerized finetuned RoBERTa model via AWS SageMaker for low-latency inference.
2. Develop browser extensions and social media plugins using the model's API for real-time credibility scoring.
3. Incorporate automation frameworks to flag low-confidence predictions for human review.

